

DS4420 Final Project

Yifan Wang, Siyuan Chen

2026-04-14

Bayesian Regression for Spotify Popularity

```
# Load packages
library(rstanarm)
library(bayesplot)
library(ggplot2)

# Read data
data <- read.csv("dataset.csv")

# Keep only the variables needed for the Bayesian model
df <- data[, c("popularity", "danceability", "energy", "tempo", "loudness", "valence")]

# Remove missing values
df <- na.omit(df)

# Standardize predictors
predictor_cols <- c("danceability", "energy", "tempo", "loudness", "valence")
df[predictor_cols] <- scale(df[predictor_cols])

# Fit Bayesian regression model
bayes_model <- stan_glm(
  popularity ~ danceability + energy + tempo + loudness + valence,
  data = df,
  family = gaussian(),
  chains = 4,
  iter = 2000,
  seed = 123,
  refresh = 0
)

# Model summary
summary(bayes_model)
```

```
##
## Model Info:
## function:      stan_glm
## family:        gaussian [identity]
## formula:       popularity ~ danceability + energy + tempo + loudness + valence
## algorithm:     sampling
## sample:        4000 (posterior sample size)
## priors:        see help('prior_summary')
## observations:  114000
## predictors:    6
##
## Estimates:
##              mean    sd   10%   50%   90%
## (Intercept)  33.2    0.1  33.2  33.2  33.3
## danceability  1.2    0.1   1.1   1.2   1.3
## energy       -1.7    0.1  -1.8  -1.7  -1.5
## tempo         0.4    0.1   0.3   0.4   0.5
## loudness      2.5    0.1   2.4   2.5   2.6
## valence      -1.8    0.1  -1.9  -1.8  -1.7
## sigma        22.2    0.0  22.1  22.2  22.2
##
## Fit Diagnostics:
##              mean    sd   10%   50%   90%
## mean_PPD  33.2    0.1  33.1  33.2  33.4
##
## The mean_ppd is the sample average posterior predictive distribution of the outcome variable
## (for details see help('summary.stanreg')).
##
## MCMC diagnostics
##              mcse Rhat n_eff
## (Intercept)  0.0   1.0  1486
## danceability  0.0   1.0  4063
## energy        0.0   1.0  2790
## tempo         0.0   1.0  5731
## loudness      0.0   1.0  2807
## valence       0.0   1.0  3647
## sigma        0.0   1.0  3228
## mean_PPD      0.0   1.0  2315
## log-posterior 0.1   1.0  1551
##
## For each parameter, mcse is Monte Carlo standard error, n_eff is a crude measure of effective
## sample size, and Rhat is the potential scale reduction factor on split chains (at convergence
## Rhat=1).
```

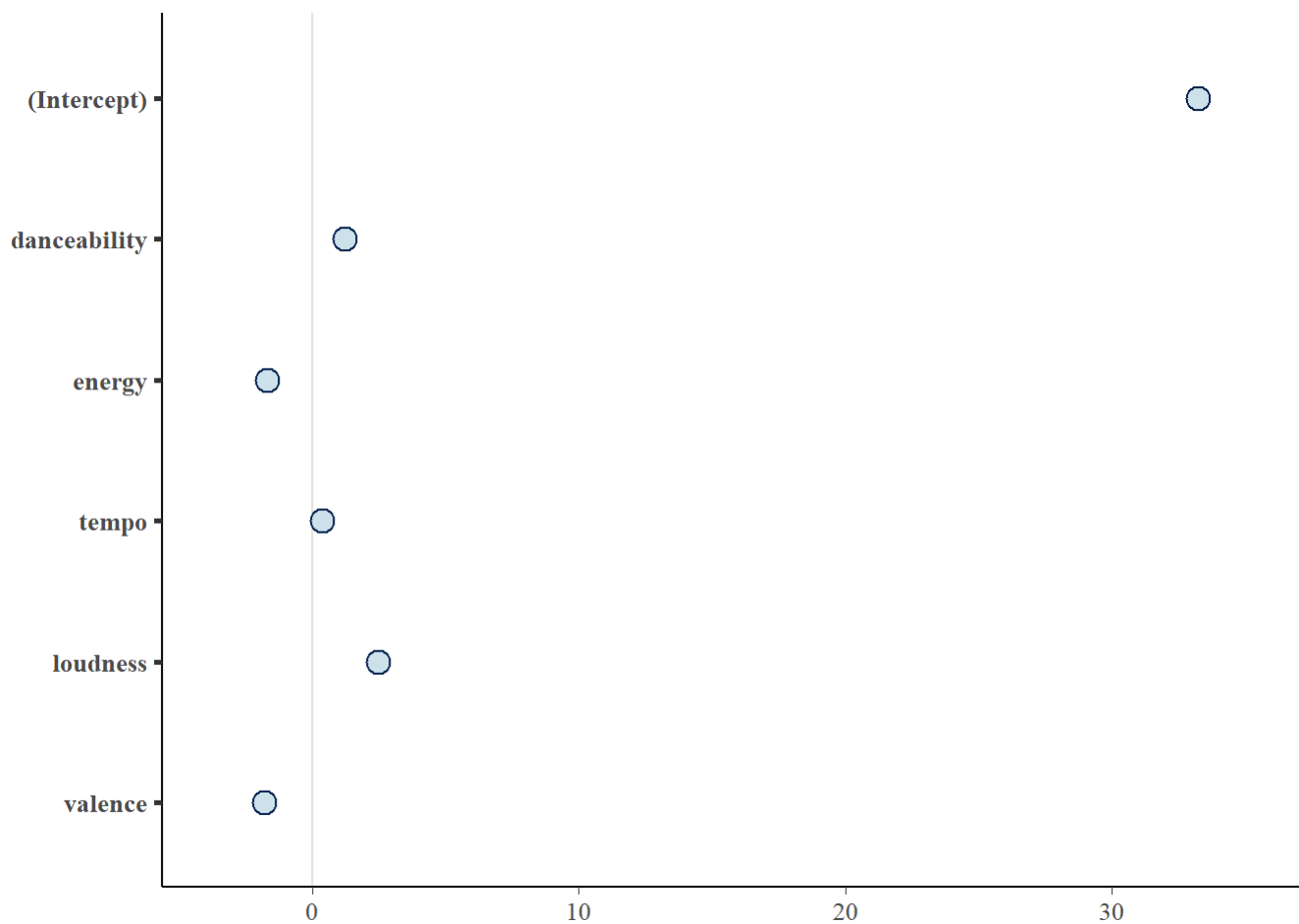
```
# 95% credible intervals
ci <- posterior_interval(bayes_model, prob = 0.95)
print(ci)
```

```
##                2.5%      97.5%
## (Intercept)  33.1092966 33.3728500
## danceability  1.0866201  1.3973424
## energy       -1.8769371 -1.4720531
## tempo        0.2514984  0.5165489
## loudness     2.2997594  2.7103400
## valence      -1.9497952 -1.6357636
## sigma        22.0923589 22.2734321
```

```
# Coefficients (posterior means)
coef_values <- coef(bayes_model)
print(coef_values)
```

```
## (Intercept) danceability      energy      tempo      loudness      valence
##  33.2399768    1.2467316   -1.6838119    0.3818518    2.5038176   -1.7929121
```

```
# Plot 1: Coefficient intervals
mcmc_intervals(
  as.matrix(bayes_model),
  pars = c("(Intercept)", "danceability", "energy", "tempo", "loudness", "valence")
)
```

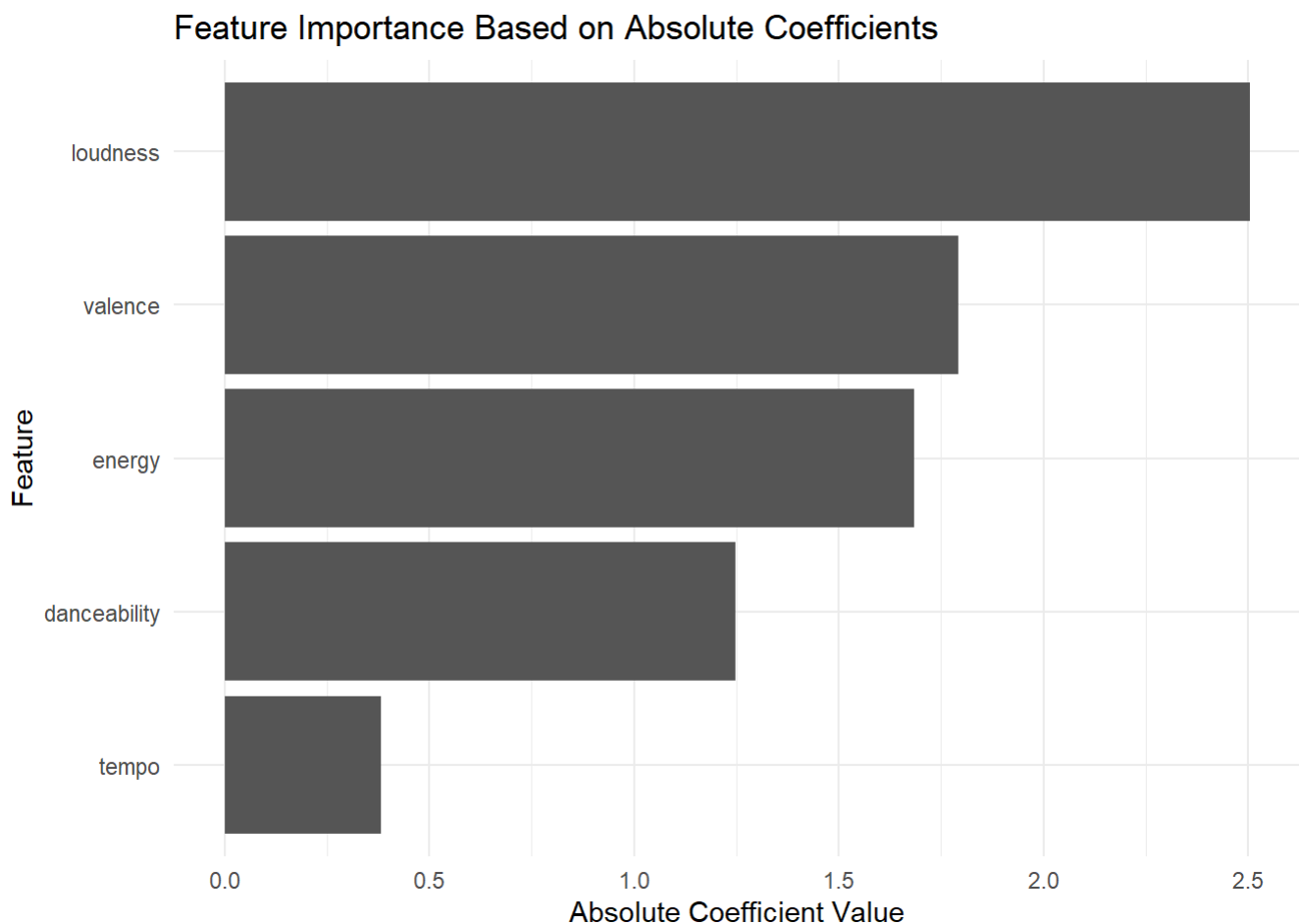


```

# Plot 2: Feature importance using absolute coefficient values
coef_df <- data.frame(
  feature = names(coef_values)[-1],
  value = abs(coef_values[-1])
)

ggplot(coef_df, aes(x = reorder(feature, value), y = value)) +
  geom_col() +
  coord_flip() +
  labs(
    title = "Feature Importance Based on Absolute Coefficients",
    x = "Feature",
    y = "Absolute Coefficient Value"
  ) +
  theme_minimal()

```



```

# Save plots
# Coefficient interval plot
png("bayesian_coefficient_intervals.png", width = 900, height = 600)
mcmc_intervals(
  as.matrix(bayes_model),
  pars = c("(Intercept)", "danceability", "energy", "tempo", "loudness", "valence")
)
dev.off()

```

```

## png
## 2

```

```
# Feature importance plot
p <- ggplot(coef_df, aes(x = reorder(feature, value), y = value)) +
  geom_col() +
  coord_flip() +
  labs(
    title = "Feature Importance Based on Absolute Coefficients",
    x = "Feature",
    y = "Absolute Coefficient Value"
  ) +
  theme_minimal()

ggsave("bayesian_feature_importance.png", plot = p, width = 9, height = 6)
```

Bayesian Interpretation

From the model results, we can see that different audio features have different effects on song popularity.

Loudness and danceability have positive effects, which means songs that are louder and more danceable tend to be more popular. On the other hand, energy and valence show negative relationships with popularity, suggesting that higher values in these features do not always lead to higher popularity. Tempo has a positive effect, but its impact is relatively small compared to the other features.

The 95% credible intervals for all variables do not include zero, which suggests that these features have meaningful effects on popularity. Overall, loudness appears to be the most important feature based on its larger coefficient value.

Bayesian Model Section

We used a Bayesian regression model in R to predict Spotify track popularity based on several audio features, including danceability, energy, tempo, loudness, and valence.

One advantage of using a Bayesian approach is that it not only gives us coefficient estimates, but also shows the uncertainty of each estimate through credible intervals. This makes it easier to understand how each feature affects popularity.

Before training the model, we standardized all input features so that they are on the same scale and easier to compare.